SHERLOCK

Security Review For Superlend

Collaborative Audit Prepared For:	**Superlend**
Lead Security Expert(s):	**0x3b** **santipu_**
Date Audited:	**October 1 - October 8, 2025**

Introduction

This security review focused on the Looping Vault which has been upgraded with a modular architecture introducing three core components: Accountant, Deposit Manager, and Withdraw Manager.

- The Accountant Module introduces a stateful universal accountant with distributed stateless plugins for portfolio tracking and fee accounting.
- The Deposit Manager now supports atomic deposits, partial fills, cancellations, and non-socialized entry fees.
- The Withdraw Manager enables partial withdrawals, slippage-based priority queues, and non-socialized exit fees.

Together, these upgrades enhance vault transparency, flexibility, and user control.

Scope

Repository: Superlend/superloop-core-contracts

Audited Commit: fb33e7d44e708cba9d7a6628e632465aa3a04ef0

Final Commit: c87f2fc34355f8bf5d14e9d9b4a35e53e96d1636

Files:

- src/common/DataTypes.sol
- src/core/Accountant/universalAccountant/UniversalAccountantBase.sol
- src/core/Accountant/universalAccountant/UniversalAccountant.sol
- src/core/DepositManager/DepositManagerBase.sol
- src/core/DepositManager/DepositManager.sol
- src/core/lib/AaveV3AccountantPluginStorage.sol
- src/core/lib/DepositManagerStorage.sol
- src/core/lib/SuperloopStorage.sol
- src/core/lib/UniversalAccountantStorage.sol
- src/core/lib/WithdrawManagerStorage.sol
- src/core/Superloop/SuperloopActions.sol
- src/core/Superloop/SuperloopBase.sol
- src/core/Superloop/Superloop.sol
- src/core/Superloop/SuperloopVault.sol
- src/core/WithdrawManager/WithdrawManagerBase.sol

- src/core/WithdrawManager/WithdrawManager.sol
- src/helpers/MigrationHelper.sol
- src/helpers/PausableUpgradeableEnhanced.sol
- src/helpers/VaultRouter.sol
- src/modules/aave/AaveV3ActionModule.sol
- src/modules/callback/DepositManagerCallbackHandler.sol
- src/modules/callback/WithdrawManagerCallbackHandler.sol
- src/modules/dex/UniversalDexModule.sol
- src/modules/fallback/AaveV3PreliquidationFallbackHandler.sol
- src/modules/helper/UnwrapModule.sol
- src/modules/helper/WrapModule.sol
- src/modules/stake/hyperliquid/HyperbeatStakingModule.sol
- src/modules/stake/hyperliquid/HyperliquidStakeModule.sol
- src/modules/stake/hyperliquid/KinetiqStakeModule.sol
- src/plugins/Accountant/AaveV3AccountantPluginBase.sol
- src/plugins/Accountant/AaveV3AccountantPlugin.sol

Final Commit Hash

c87f2fc34355f8bf5d14e9d9b4a35e53e96d1636

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
2	6	15

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Unfair performance fee after migration

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/40

Summary

When a vault is migrated from V1 to V2, the treasury immediately earns an unfair performance fee.

Vulnerability Detail

During vault migration, the accountant module on the new vault does not update the `lastRealizedFeeExchangeRate` variable. As a result, it assumes the previous exchange rate was zero. This causes the module to interpret the current exchange rate as pure profit and calculate a large performance fee accordingly, transferring an excessive amount to the treasury.

Example with XTZ Loop Vault:

1. At the time of writing, the XTZ Loop Vault has the following state:
    - totalAssets ≈ `576_668e18`
    - totalSupply ≈ `~541_190e20` (2 extra decimals due to offset)
2. When the vault is migrated to V2, the accountant module will calculate the performance fee (15%) as follows:
    - exchangeRate = `1e18 * 576_668e18 / 541_190e20` ≈ `0.011e18`
    - interestAssets = `(exchangeRate * totalSupply - 0) * 0.15e4 / (1e4 * 1e18)` ≈ `86_500e18`
3. The vault then mints shares to the treasury worth approximately 86,500 wXTZ, equal to 15% of the total vault assets.

Impact

Post-migration, the vault mints shares to the treasury valued at 15% (in case of XTZ Loop Vault) of its total assets, resulting in user losses.

Even if the treasury reverses the fee by returning the assets, the temporary fluctuations in the exchange rate create an arbitrage opportunity for MEV bots, allowing them to extract value from users while the correction is made.

Code Snippet

Tool Used

Manual Review

Recommendation

Update the migration flow to explicitly set the `lastRealizedFeeExchangeRate` variable on the accountant module associated with the new vault.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/77325117b0fc31ca25b90aed6fd39ab3058c6ec8

Issue H-2: Flash loan callback can be tricked to steal all funds from V1 vault

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/54

Summary

On `MigrationHelper`, the Aave callback (`executeOperation`) can be tricked to steal all funds from the V1 vaults.

Vulnerability Detail

When vaults are going to be migrated, the flow is as follows:

1. `MigrationHelper` is added as a privileged address in the old vault.
2. `MigrationHelper` is added as deposit manager and operator in the new vault.
3. `MigrationHelper::migrate` is called to execute the actual migration.

The issue here is that if the following flow is not executed atomically (within the same transaction), then an attacker can execute a malicious transaction between steps 1 and 3 that could steal all the funds from the old vault.

The root cause is that the Aave callback doesn't check that the initiator of the flash loan is the `MigrationHelper` contract itself:

```
function executeOperation(address asset, uint256 amount, uint256 premium, address,
↪  bytes calldata params)
    external
    returns (bool)
{
    require(msg.sender == address(POOL), "Only pool can call flashloan callback");

    // MISSING INITIATOR VALIDATION HERE!!

    // ...
}
```

Due to this missing validation, an attacker can do the following to drain the vault:

1. Call `Pool.flashLoanSimple` setting the `receiver` as the `MigrationHelper` contract and crafting custom data in the `params` argument.
2. Aave will call `MigrationHelper::executeOperation`, and this function will decode the `params` argument and use that data to execute the migration.

3. Then, the `MigrationHelper` contract will withdraw all funds from the old vault and deposit them into a vault that is controlled by the attacker.
4. Finally, the attacker can withdraw all funds from his controlled vault, effectively stealing everything from the legitimate vault users.

Impact

Anyone can steal all funds from the V1 vaults.

Code Snippet

Tool Used

Manual Review

Recommendation

It's recommended to add a simple validation check on `executeOperation` that ensures the `initiator` argument is the `MigrationHelper` contract itself.

Note: The same issue exists on `AaveV3CallbackHandler`, but that contract is currently out of scope for this audit. However, we still highly recommend adding an initiator check there as well.

Issue M-1: Deposit manager does not realize the performance fee

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/32

Summary

The deposit manager doesn't realize the performance fee, which results in shares being minted at a different ratio compared to a normal deposit.

Vulnerability Detail

In normal vault actions, the `_realizePerformanceFee` function is always called first.

```
    function deposit(uint256 assets, address receiver) public override nonReentrant
↪   whenNotPaused returns (uint256) {
        // realize performance fee
        _realizePerformanceFee();
```

However, when resolving deposits with the deposit manager, `resolveDepositRequests` first triggers `executeDeposit`.

```
        IDepositManagerCallbackHandler(cache.vault).executeDeposit(data.amount,
↪   callbackExecutionData);
```

It then mints vault shares to each user, missing the performance fee update.

```
            if (sharesToMint != 0) {
                $.depositRequest[currentId].sharesMinted =
↪   currentRequest.sharesMinted + sharesToMint;
                ISuperloop(cache.vault).mintShares(currentRequest.user,
↪   sharesToMint);
            }
```

Impact

Shares minted by the deposit manager will have a different ratio than those minted directly from the vault.

Tool Used

Manual Review

Recommendation

Add `realizePerformanceFee` in `resolveDepositRequests`.

```
    function resolveDepositRequests(DataTypes.ResolveDepositRequestsData memory
↪   data) external onlyVault {
        (DepositManagerCache memory cache,
↪   DepositManagerStorage.DepositManagerState storage $) =
            _createDepositManagerCache();

+    ISuperloop(cache.vault).realizePerformanceFee();
```

Discussion

santipu03

Valid issue, commented yesterday here. The same will happen with WithdrawManager.

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/a38a7b8d8d053792955a3a19bbe67e542ea369bd

Issue M-2: Wrong order of share accrural

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/37

Summary

Wrong order of share accrual would cause slightly less fees to be generated.

Vulnerability Detail

When `_realizePerformanceFee` is called, the latest exchange rate is first set with `setLastRealizedFeeExchangeRate`, but then shares are minted to the treasury, which in turn would lower the exchange rate:

```
function _realizePerformanceFee() internal {
        // ...
        uint256 sharesToMint = _getPerformanceFeeAndShares(exchangeRate,
↪  $.accountant, decimals);


↪  IAccountantModule($.accountant).setLastRealizedFeeExchangeRate(exchangeRate,
↪  totalSupply());

        if (sharesToMint > 0) {
            // mint the shares to the treasury
            _mint($.treasury, sharesToMint);

            emit PerformanceFeeRealized(sharesToMint, $.treasury);
        }
```

This would lower the total generated fees as value would now need to accrue for the current lower rate to reach the one set before minting the shares.

Example

1. 1000 shares and 1200 assets, with a 10% fee to the treasury.
2. Share value rose from 1 to 1.2 (0.2% fee to the treasury).
3. The new rate is set to 1.2.
4. 16.9 shares are minted, and the new `totalSupply` is 1016.9.
5. The new actual rate is $1200/1016.9 = 1.18$.
6. The system makes 0 profit from the share value increase until it hits 1.2.

Variables:

- totalA = $1200e18$
- totalS = $1000e18$
- currentExRate = 1.2
- lastExRate = 1
- treasury fee = 10% => assets = $20e18$

$$\textit{denominator} = 1200e18 - 20e18 + 1 = 1180e18 + 1$$
$$\textit{numerator} = (1200e18 + 1) \cdot (1000e18 + 10e12) - ((1000e18 + 10e12) \cdot (1180e18 + 1)) = 2e40$$
$$\textit{result} = \frac{2e40}{1180e18 + 1} = 16.9e18$$

Impact

The treasury fee would be lowered.

Code Snippet

Tool Used

Manual Review

Recommendation

Change the order to first mint the shares, then set the rate. Make sure to recalculate the new rate before setting it.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/0d46b6af6a6c07994ee4fddc2d929818dc08a3f3

Issue M-3: Disable performance fee by direct donation on first deposit

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/45

Summary

When the first deposit happens, an attacker can make a direct donation to the vault, which imbalances the exchange rate and effectively disables the performance fee due to rounding.

Vulnerability Detail

When a vault is initialized, the first deposit must be executed through `DepositManager` due to the `_getCashReserveShortfall` check on direct deposits. If an attacker transfers some assets directly to the vault before the first deposit is executed, the shares minted in the first deposit will be much lower than expected, making the exchange rate be initialized much lower as well.

When the total supply of shares is super low, and the total assets are high, it will cause the calculations for the performance fee shares to mint to be rounded down to zero.

PoC

First, pin the fork to a specific block for consistency. Go to `TestBase.sol` and add the following line:

```
    function setUp() public virtual {
        vm.createSelectFork("etherlink");
+       vm.rollFork(27357101);

        // ...
    }
```

Then, paste and run the following test within `DepositManager.t.sol`:

```
function test_donation_first_deposit() public {
    // The first deposit is requested
    uint256 depositAmount = 10e18;
    _makeDepositRequest(depositAmount, user1, true);

    // A griefer transfers some assets directly to the vault
    deal(XTZ, address(superloop), 100e18);
```

```
// Griefer realizes performance fee to update rate without minting shares to
↪ treasury
superloop.realizePerformanceFee();
assertEq(superloop.balanceOf(treasury), 0);

// build the operate call for the deposit of 10e18
uint256 supplyAmount = 15 * STXTZ_SCALE;
uint256 borrowAmount = 6 * XTZ_SCALE;
uint256 swapAmount = borrowAmount + depositAmount;
uint256 supplyAmountWithPremium = supplyAmount + (supplyAmount * 1) / 10000;

DataTypes.ModuleExecutionData[] memory moduleExecutionData = new
↪ DataTypes.ModuleExecutionData[](3);
moduleExecutionData[0] = _supplyCall(ST_XTZ, supplyAmount);
moduleExecutionData[1] = _borrowCall(XTZ, borrowAmount);
moduleExecutionData[2] = _swapCallExactOutCurve(XTZ, ST_XTZ, XTZ_STXTZ_POOL,
↪ swapAmount, supplyAmountWithPremium, XTZ_STXTZ_SWAP);

DataTypes.ModuleExecutionData[] memory intermediateExecutionData = new
↪ DataTypes.ModuleExecutionData[](1);
intermediateExecutionData[0] = _flashloanCall(ST_XTZ, supplyAmount,
↪ abi.encode(moduleExecutionData));

DataTypes.ModuleExecutionData[] memory finalExecutionData = new
↪ DataTypes.ModuleExecutionData[](1);
finalExecutionData[0] = _resolveDepositRequestsCall(XTZ, depositAmount,
↪ abi.encode(intermediateExecutionData));

vm.prank(admin);
superloop.operate(finalExecutionData);

// Total supply is 110 wei and total assets is 109.5e18, so the exchange rate
↪ is super imbalanced
assertEq(superloop.totalSupply(), 110);
assertEq(superloop.totalAssets(), 109.559034854239289379e18);

// Then, we simulate the totalAssets increasing from 109.5 to 120 (9.5%
↪ increase)
deal(XTZ, address(this), 10.5e18);
IERC20(XTZ).transfer(address(superloop), 10.5e18);

// Total assets is now ~120e18
assertEq(superloop.totalAssets(), 120.059034854239289379e18);

// Now, we realize the performance fee
superloop.realizePerformanceFee();
// The treasury hasn't received any shares due to rounding errors due to the
↪ imbalanced exchange rate
assertEq(superloop.balanceOf(treasury), 0);
```

```
}
```

As shown by the PoC, when the first deposit is 10e18, the attacker can donate 100e18 assets to effectively disable the performance fee forever due to rounding.

Even though it's true that this exploit requires an initial donation, the loss of fees for the treasury over time is much higher than the initial cost of the attack.

Impact

Making a direct donation when the vault is initialized causes the exchange rate to be highly imbalanced, leading to the performance fee calculations to round down to zero forever.

Code Snippet

Tool Used

Manual Review

Recommendation

It's recommended to create a special method on the vault to execute the first deposit instantly so that the exchange rate is initialized with a significant amount of assets (e.g., 1e18). It's critical that this first deposit is executed in the same txn as the vault is deployed to avoid direct donations beforehand.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/1cc37330e4cced5caa618f44132bda9f609b32ad

santipu03

The fix has resulted in the creation of a new function `seed`, which allows the vault admin to execute the first deposit when the vault has been deployed. However, the effectiveness of this fix relies on the condition that `seed` MUST be called on the same transaction as the vault is deployed. Otherwise, an attacker could inject a malicious transaction in between to exploit this issue.

Issue M-4: Stuck funds in `VaultRouter`

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/47

Vulnerability Detail

The `VaultRouter` contract serves as an entry point for users who wish to deposit into a vault using a token different from the vault's deposit token. It interacts with the DEX module to perform the token swap and subsequently makes the deposit on the user's behalf.

The DEX module supports two types of swaps:

1. Swaps where the exact input is specified.
2. Swaps where the exact output is specified.

In the latter cases, any unused input tokens are returned to the caller, which in this context is the router.

However, the router does not handle or forward these unspent tokens, resulting in them being permanently held within the contract.

Impact

Unspent tokens remain stuck in the `VaultRouter` contract, leading to a small loss of user funds for each exact output swaps.

Code Snippet

Tool Used

Manual Review

Recommendation

Update `VaultRouter::depositWithToken` to return any unspent input tokens to the original caller.

Alternatively, if the router is intended to support only exact-input swaps, clearly document this behavior to ensure integrators are aware of the limitation.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/00f6b16ae25bcd6d0ac767e3c22044488e326dff

Issue M-5: Pre-liquidations open arbitrage vectors

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/51

Summary

All preliquidations cause the vault exchange rate to decrease, opening arbitrage vectors for clever users that can avoid the loss at the expense of the others.

Vulnerability Detail

During a preliquidation, the vault ends up with fewer assets than before due to the incentive factor. This causes the exchange rate in the vault to decrease due to a pre-liquidation, and that can be weaponized by a clever user to avoid the loss.

Example scenario:

1. Initial exchange rate is 1.5
2. The vault is eligible for preliquidation
3. Before the preliquidation, a user makes an instant withdrawal from the vault, withdrawing the maximum amount allowed.
    - For example, let's say that it withdraws 1,000 shares and gets 1,500 USDC.
4. Then, the vault is preliquidated, causing the exchange rate to decrease to 1.4 due to the incentive factor.
5. After that, that user deposits again the 1,500 USDC, but now gets ~1,071 shares due to the decrease in exchange rate.
6. At the end, the user has avoided the loss by withdrawing and depositing again.

This sandwich attack can be performed with any action that alters the exchange rate, either upwards or downwards.

The docs state that there are more actions that can cause this, so this scenario above is just one example of many where this can be exploited.

Impact

Exchange rate alterations can be arbitraged by clever users or bots.

This can happen as long as there are some idle assets in the vault for instant withdrawals.

Code Snippet

Tool Used

Manual Review

Recommendation

I think the best mitigation here would be to introduce an entry and exit fee only for instant deposits and withdrawals. These fees would disincentivize any kind of arbitrage in the vault by making it unprofitable.

Issue M-6: Migrations can be griefed by increasing the amount of share holders

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/55

Summary

The migration process from V1 vaults to V2 vaults can be griefed by making a ton of deposits with a tiny value on behalf of random unique addresses.

Vulnerability Detail

The migration process requires the admins to pass a list of all addresses that hold shares of the V1 vault. There's an explicit check that enforces that:

```
function _validateUserBalancesWithTotalSupply(VaultStateData memory vaultState)
 ↪  internal pure {
    uint256 calculatedTotalSupply = 0;
    for (uint256 i = 0; i < vaultState.balances.length; i++) {
        calculatedTotalSupply += vaultState.balances[i];
    }
    require(calculatedTotalSupply == vaultState.totalSupply, "Calculated total
     ↪  supply does not match total supply");
}
```

However, this strict requirement allows a griefer to prevent the migration by increasing the addresses that hold vault shares. This can be executed by making a ton of deposits with little to no value on behalf of random, unique addresses.

If we check the fork tests on migrations (`MigrationTest::test_migration`), we see that migrating a vault with only 115 holders spends more than 6M gas units. If a griefer were to make 10,000 transactions giving a tiny amount of shares to random addresses, the amount of gas spent would likely surpass 30M, which is the maximum gas that can be spent on a block in the Etherlink chain. This would prevent the migration txn from ever executing.

Impact

The migration process can be blocked due to OOG errors.

Code Snippet

Tool Used

Manual Review

Recommendation

I believe the best mitigation here would be to remove the check on `_validateUserBalancesWithTotalSupply`.

If a griefer were to execute this exploit, then the protocols admins could just choose only to migrate the addresses that have a non-trivial amount of shares, allowing to exclude puppet addresses that were funded by the grierfer.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/d77550d4fb36b547a6d0b8ed115404efa74eda7c

Issue L-1: When canceling a deposit, funds are sent to the wrong recipient

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/34

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When a deposit is canceled, the funds are sent to the user who would have received the deposit, not the user who initiated it.

Vulnerability Detail

Users can make deposits on behalf of other users.

However, when these deposits are canceled, the funds are sent to the address who was originally intended to receive the deposit:

```
    function _handleCancelDepositRequest(
        DepositManagerCache memory cache,
        DepositManagerStorage.DepositManagerState storage $,
        uint256 id,
        DataTypes.DepositRequestData memory _depositRequest
    ) internal returns (uint256) {

        //...
        SafeERC20.safeTransfer(IERC20(cache.asset), _depositRequest.user,
↪   amountToRefund);
```

This can be dangerous as that address may not be able to handle these tokens in the first place (can be reason to cancel the deposit).

Example:

1. Contract1 makes a deposit of specific tokens for Contract2.
2. The admin of Contract2 sees that their contract does not support these tokens, so they cancel the deposit.
3. The deposit is canceled, but the tokens still end up in Contract2.

While cases like this are rare, it is better to handle or at least document this issue.

Impact

Contracts might get stuck with tokens they cannot handle.

Tool Used

Manual Review

Recommendation

Send the deposit back to the user who originally made it.

Discussion

priyam-anand

Sending the funds back to the originator will break our business logic. We will have to set it back to the user who was supposed to be the receiver. We will make sure to document this behaviour

Issue L-2: Deposit requests can be DOSed

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/35

Summary

A malicious user can submit a bad deposit request on behalf of another user, effectively blocking that user from submitting their own deposit request until the first one is either processed or canceled.

Vulnerability Detail

We can request deposits for other users and there is no minimum required amount, besides that the shares received must be at least 1 wei (i.e. min is 1 or 2 wei).

When making the actual deposit, the last one must be processed or canceled.

```
        if (id != 0) {
            bool isPending = _depositRequest.state ==
↪   DataTypes.RequestProcessingState.UNPROCESSED;
            bool isUnderProcess = _depositRequest.state ==
↪   DataTypes.RequestProcessingState.PARTIALLY_PROCESSED;

            if (isPending || isUnderProcess) {
                revert(Errors.DEPOSIT_REQUEST_ACTIVE);
            }
        }
```

This would allow users to make deposits of 1 or 2 wei for other users and block them from requesting a deposit. The only way for anyone to schedule a deposit would be to either cancel this bad request (wasting time and gas) or just wait it out to be executed.

Example:

1. The vault is full, so the only way to make deposits is from the deposit manager.
2. The average waiting time for your deposit to be executed is 2 days.
3. Alice doesn't like Bob so she makes a 2 wei deposit for him.
4. Bob can't deposit, he needs to cancel the bad deposit before he can.

Impact

Some users might be temporarily DoS from making requests.

Tool Used

Manual Review

Recommendation

Consider adding a minimum required amount in order to request a deposit.

Issue L-3: Some imports are not needed

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/36

Summary

Remove unused imports

Vulnerability Detail

Some imports are not used anywhere and can be removed. An example is `console` and `IERC4626` inside the `WithdrawManager`:

```
import {IERC4626} from "openzeppelin-contracts/contracts/interfaces/IERC4626.sol";
import {console} from "forge-std/console.sol";
```

Impact

Not a dangerous issue; it just wastes a bit of gas on deployment.

Tool Used

Manual Review

Recommendation

Check every contract for any unneeded imports and remove them.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/99e32428b492f1ed0249404c0705f8bd1552cb3e

Issue L-4: Updating roles can revoke privileged access for an address that holds multiple roles

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/38

Summary

When updating roles, the current implementation revokes the privileged status of the old address before assigning the new one. This can unintentionally remove an address's privileged status if it holds multiple roles.

Vulnerability Detail

The current flow for updating a vault role is as follows::

1. Revoke privileged status from the address currently holding the role.
2. Update the role to the new address.
3. Grant privileged status to the new address.

```
function setVaultOperator(address vaultOperator_) external onlyVaultAdmin {
    address currentVaultOperator =
    ↪  SuperloopStorage.getSuperloopEssentialRolesStorage().vaultOperator;

    _setPrivilegedAddress(currentVaultOperator, false);
    SuperloopStorage.setVaultOperator(vaultOperator_);
    _setPrivilegedAddress(vaultOperator_, true);
    emit VaultOperatorUpdated(currentVaultOperator, vaultOperator_);
}
```

This approach works as intended when each role is assigned to a unique address. However, if a single address holds multiple roles (e.g., operator and treasury), revoking its privileged status while updating one role removes its access entirely, even though it should retain privileges due to its remaining roles.

Impact

An address holding multiple roles may lose privileged access prematurely. This could prevent authorized users from performing privileged operations, potentially leading to system disruption or malfunction.

Code Snippet

Tool Used

Manual Review

Recommendation

Modify the role update mechanism to ensure that an address with multiple roles retains its privileged status until all associated roles have been revoked.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/52ccb2d8cbc8a4767dd7ba0460e9537cbef9d6a6

Issue L-5: Performance fee formula can be simplified

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/39

Summary

The current performance fee formula is overly complicated and can be simplified.

Vulnerability Detail

The current formula for calculating the amount of shares the treasury deserves for an amount of assets is as follows:

Terms: S = `totalSupply + 10offset` A = `totalAssets + 1` a = `assets`

Current formula:

$$shares = \frac{(A \times S) - (S \times (A - a))}{A - a}$$

Which is mathematically equivalent to:

$$shares = \frac{S \times a}{A - a}$$

Quick demonstration:

```
shares = ((A * S) - (S * (A - a)) / (A - a)
shares = ((A * S) - (S * A) - (S * a)) / (A - a)
shares = ((S * a) / (A - a)
```

Impact

The current performance fee formula is overly complicated and it's a waste of gas.

Code Snippet

Tool Used

Manual Review

Recommendation

Update that performance fee formula to match the simplified version.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/a36a1a7a8b088175a16fe557630d392c49d10eff

Issue L-6: Superloop vault doesn't strictly follow EIP-4626

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/41

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Superloop vault doesn't strictly follow EIP-4626

Vulnerability Detail

According to EIP-4626, the functions `maxDeposit` and `maxMint` should always return the amount of assets or shares that the vault will accept on a deposit/mint, and never overestimate:

EIP-4626:

> **maxDeposit** MUST return the maximum amount of assets deposit would allow to be deposited for receiver and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).

This means that if the vault were to have strict compliance with EIP-4626, then the functions `maxDeposit` and `maxMint` should also account for the cash reserves, as this is a limitation that can make normal deposits revert.

Additionally, the functions `maxWithdraw` and `maxRedeem` should account for the amount of idle assets that are currently in the vault. These two functions currently default to their OZ implementation, which differs from the current vault behavior:

```
/** @dev See {IERC4626-maxWithdraw}. */
function maxWithdraw(address owner) public view virtual returns (uint256) {
    return _convertToAssets(balanceOf(owner), Math.Rounding.Floor);
}

/** @dev See {IERC4626-maxRedeem}. */
function maxRedeem(address owner) public view virtual returns (uint256) {
    return balanceOf(owner);
}
```

Impact

Broken strict compliance with EIP-4626

Code Snippet

Tool Used

Manual Review

Recommendation

Consider updating the affected functions to follow EIP-4626.

If the current behavior is desired, it's recommended to document the fact that the Superloop vault doesn't strictly follow EIP-4626 so that external integrators are aware of it.

Discussion

priyam-anand

This would be the expected and correct behaviour for our use case. We will make sure to document this.

Issue L-7: 2-decimal vault assets cause exchange rate precision loss

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/42

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When the vault's main asset has only 2 decimals, the exchange rate lacks decimal precision, resulting in significant rounding errors. This allows for vault arbitrage and prevents the performance fee from being charged unless asset value roughly doubles.

Vulnerability Detail

Consider the following scenario:

1. Initial state:
    - totalAssets = `10_000e2` (e.g., 10,000 GUSD)
    - totalSupply = `10_000e4` (2 more decimals due to offset)
    - exchangeRate = `1e2 * (10_000e2 + 1) / (10_000e4 + 100)` = **1**
2. The assets in the vault increase by 50%
    - totalAssets = `15_000e2`
    - totalSupply = `10_000e4` (same as before)
    - exchangeRate = `1e2 * (15_000e2 + 1) / (10_000e4 + 100)` = **1**

Despite a 50% asset increase, the exchange rate remains 1 due to rounding, meaning no performance fee is triggered.

GUSD is a known asset with 2 decimals, though it is not currently deployed on Etherlink or Hyperliquid, Superlend's primary chains. However, similar assets could be introduced in the future, leading to this issue.

Impact

Vaults using assets with only 2 decimals will experience severe exchange rate rounding. This prevents accurate fee realization and creates arbitrage opportunities for MEV bots to extract value from users. The vault may become unable to collect performance fees unless the asset value increases significantly.

Code Snippet

Tool Used

Manual Review

Recommendation

Avoid using assets with only 2 decimals in vaults. If support for such assets becomes necessary in the future, implement a mechanism to preserve exchange rate precision despite limited decimal places.

Issue L-8: Uninitialized `resolutionIdPointer`

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/43

Vulnerability Detail

On `DepositManager`, the variable `resolutionIdPointer` should be initialized to 1 to match the first request ID.

Impact

When the first deposit request is going to be resolved, the first loop iteration will do nothing because there's no request with an ID of 0, leading to a waste of gas.

Code Snippet

Tool Used

Manual Review

Recommendation

On `__SuperloopDepositManager_init`, initialize the variable `resolutionIdPointer` to match the variable `nextDepositRequestId`.

Do the same on `WithdrawManager`.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/9a7fc5b0125fc49cbb27d9df6d56e354b512fbbb

Issue L-9: Freeze operations can revert

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/46

Vulnerability Detail

When `setFrozen` is called, the vault admin can freeze or unfreeze the contract. However, when freezing, the contract is automatically paused, and when unfreezing, it's unpaused as well:

```
function _freeze() internal virtual whenNotFrozen {
    PausableStorage storage $ = _getPausableStorage();
    $._frozen = true;

    _pause();

    emit Frozen(_msgSender());
}
```

However, the issue here is that the function `_pause` will revert if the contract is already paused. Also, `_unpause` reverts if already not paused.

This can be a problem when the admin tries to freeze when the contract is already paused, as this will cause the txn to revert. Similarly, the admin won't be able to unfreeze if the contract is not paused, but this scenario is much less probable.

Impact

Temporary DoS.

Code Snippet

Tool Used

Manual Review

Recommendation

It's recommended to update `_freeze` so that it only pauses the contract when it's not already paused. The same with `unfreeze`.

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/eb3ed53c50d3cb7af1030639ba03869c79c27016

Issue L-10: Underflow on `AaveV3AccountantPlugin`

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/48

Vulnerability Detail

On `AaveV3AccountantPlugin::getTotalAssets`, there's a potential underflow that would happen when the value of the borrowed assets is higher than the value of the supplied assets:

```
// convert to base asset
uint256 totalAssetsInBaseAsset = (positiveBalance - negativeBalance) /
↪  baseAssetPrice;
```

Even though a position with more debt value than collateral value should have been liquidated, there's an edge case that can bypass this limitation.

If an external actor were to deposit some collateral in Aave on behalf of the vault using an asset that is not within the `lendAssets` array, we could arrive at a situation where the position on Aave is healthy, but the `getTotalAssets` function reverts due to this underflow. This would cause a DoS on `totalAssets` that would brick all deposits and withdrawals on the vault until it's fixed.

We acknowledge that the preconditions for such a scenario are highly unrealistic under the current assumptions, but these could change in the future. Also, there's always the possibility that this little issue could become the precondition for a much bigger exploit that we aren't aware of.

Impact

DoS on deposits and withdrawals in the vault.

Code Snippet

Tool Used

Manual Review

Recommendation

Because the fix to this issue is trivial, we recommend fixing it even though we haven't found a realistic way to exploit this flaw.

```
        // convert to base asset
-       uint256 totalAssetsInBaseAsset = (positiveBalance - negativeBalance) /
↪  baseAssetPrice;

+       uint256 netBalance = positiveBalance > negativeBalance ? positiveBalance -
↪  negativeBalance : 0;
+       uint256 totalAssetsInBaseAsset = netBalance / baseAssetPrice;
```

Discussion

priyam-anand

Fixed in : https://github.com/Superlend/superloop-core-contracts/commit/b1aa5c162d45ccd6fd5a9892d8b0fc2635375696

Issue L-11: Wrong check on `_validatePreLiquidationParams`

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/49

Vulnerability Detail

The `_validatePreLiquidationParams` function in the `AaveV3PreliquidationFallbackHandler` contract is intended to validate preliquidation parameters to ensure they fall within acceptable bounds. However, one of the checks is incorrect:

```
    require(preLiquidationParams_.preCF1 <= preLiquidationParams_.preCF2,
    ↪  Errors.PRELIQUIDATION_LCF_DECREASING);
>>  require(preLiquidationParams_.preCF1 <= WadRayMath.WAD,
↪   Errors.PRELIQUIDATION_LCF_TOO_HIGH);
```

The purpose of these validations is to ensure the close factor parameters are within range. Specifically, `preCF1` is the lower bound and `preCF2` the upper bound of the close factor, which defines the portion of a debt position to be repaid during preliquidation. The close factor must not exceed `WAD` (1e18), as that would imply repaying more debt than exists.

Therefore, the second check is incorrectly applied to `preCF1`. The correct validation should ensure `preCF2 <= WAD`. Checking `preCF1 <= WAD` is redundant because it is already constrained by the earlier check to be less than or equal to `preCF2`.

Impact

While the contract is intended to be deployed by trusted parties, and the risk is minimal, the validation logic should still be correct to avoid incorrect assumptions or potential misuse.

Code Snippet

Tool Used

Manual Review

Recommendation

Correct the validation logic for the upper bound:

```
    require(preLiquidationParams_.preCF1 <= preLiquidationParams_.preCF2,
    ↪  Errors.PRELIQUIDATION_LCF_DECREASING);
-   require(preLiquidationParams_.preCF1 <= WadRayMath.WAD,
↪   Errors.PRELIQUIDATION_LCF_TOO_HIGH);
+   require(preLiquidationParams_.preCF2 <= WadRayMath.WAD,
↪   Errors.PRELIQUIDATION_LCF_TOO_HIGH);
```

Discussion

priyam-anand

Hi @santipu03, the mentioned checks are intentional. A pre-liquidation close factor higher than 100% means that the whole position is pre-liquidatable, so using preLCF2 > 1 allows to make the position entirely pre-liquidatable at a LTV between preLltv and LLTV.

Do you see any potential security risks with this approach?

santipu03

@priyam-anand But why the need to set `preLCF2 > 1`? If you want to pre-liquidate the whole position, then setting `preLCF2 = 1` is enough.

If `preLCF2 > 1`, then `maxDebtToCover` will be higher than the total position's debt, causing that `collateralToSieze` may be higher than the total position's collateral, and that would cause a revert on the call to `pool.withdraw`.

priyam-anand

> @priyam-anand But why the need to set `preLCF2 > 1`? If you want to pre-liquidate the whole position, then setting `preLCF2 = 1` is enough.
> If `preLCF2 > 1`, then `maxDebtToCover` will be higher than the total position's debt, causing that `collateralToSieze` may be higher than the total position's collateral, and that would cause a revert on the call to `pool.withdraw`.

The idea is to enable user to set an range of LCF such that he may want to allow liquidation of entire position before the LLTV itself. If we set `preLCF2 = 1` then it will allow liquidation of of entire position only at LLTV.

santipu03

Okay, I get why `preLCF2 > 1` makes sense now.

However, to prevent over-repaying, I'd ensure that `debtToCover` is never above the position's debt, as this could cause the contract to transfer more funds from the caller than needed for repayment, resulting in some tokens getting stuck in `AaveV3PreliquidationFallbackHandler`.

Example scenario:

1. Vault's position on Aave is 100 collateral and 80 debt

2. A user is about to call `preliquidate` to increase the Health Factor of the position by repaying all debt.
3. Just before the pre-liquidation transaction, the vault admin decides to repay 10 debt tokens.
4. When the pre-liquidation is executed, the user wanted to repay 80 debt tokens, but now there are only 70 debt tokens. The contract will transfer 80 debt tokens from the caller, but only 70 will be used for repayment. The other 10 tokens will be stuck in the contract forever.

To fix the over-repayment of debt, something like this would do:

```
      // calculate how much collateral to sieze : (debtCoverUSD /
      ↪  collateralPriceUSD) * incentiveFactor
      uint256 maxDebtToCover = (borrowTokens * currentCloseFactor) /
      ↪  WadRayMath.WAD; // in tokens
-     uint256 debtToCover = Math.min(params.debtToCover, maxDebtToCover);
+     uint256 debtToCover = Math.min(Math.min(params.debtToCover, maxDebtToCover),
↪  borrowTokens);
      uint256 debtCoverUSD = (debtToCover * borrowPriceUSD) / (10 **
      ↪  borrowReserveDecimals); // in oracle decimals
```

priyam-anand

This makes sense. Although when repaying loan > borrow amount in aave, the transaction reverts, but it does makes sense to prevent the revert from happening.

Adding this check.

santipu03

> Although when repaying loan > borrow amount in aave, the transaction reverts...

That's not the case on Aave, check the following PoC for quick confirmation:

https://gist.github.com/santipu03/47518297a86ff7bd54be1510cea6ca26

Still, the mitigation fixes this issue, so problem solved!

Issue L-12: Incorrect event emission on preliquidations

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/50

Vulnerability Detail

After a preliquidation is executed, the following event is emitted:

```
// emit event
emit PreliquidationExecuted(id, params.user, user, debtToCover, collateralToSieze);
```

However, the 2nd event argument is incorrect as it should be the preliquidator, therefore, the caller of the function:

```
event PreliquidationExecuted(
    bytes32 indexed id, address preliquidator, address vault, uint256 debtToCover,
    ↪  uint256 collateralToSieze
);
```

Impact

Wrong event parameter

Code Snippet

Tool Used

Manual Review

Recommendation

```
    // emit event
-   emit PreliquidationExecuted(id, params.user, user, debtToCover,
↪   collateralToSieze);
+   emit PreliquidationExecuted(id, _msgSender(), user, debtToCover,
↪   collateralToSieze);
```

Issue L-13: Stale parameters on preliquidations when Aave forks update Liquidation Thresholds

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/52

Vulnerability Detail

In the `AaveV3PreliquidationFallbackHandler` contract, all parameters are based on the assumption that the Liquidation Threshold (LT) on the Aave forks hasn't changed since the contract was deployed.

When the Aave fork that is used decides to update the LT of an asset, then the immutable variables on `AaveV3PreliquidationFallbackHandler` will be stale, causing malfunctions on preliquidations.

For example, the parameter `preLltv` is set based on the current LT. If the LT were to be updated upwards or downwards, then the calculated quotient (`currentQuotient`) wouldn't represent the real quotient due to the updated LT, causing the Inceitive Factor and Close Factor to also be stale.

Impact

When the LT in the Aave fork used is updated, preliquidations will miscalculate parameters and won't function as expected. For example, maybe the contract allows preliquidations when it shouldn't, causing unfair exchange rate decreases due to the incentive factor.

Code Snippet

Tool Used

Manual Review

Recommendation

I think the best mitigation should be to always revert a preliquidation when the immutable variable `Lltv` doesn't match the real LLTV on the Aave fork.

This way, the protocol admins would be aware that the LT on the Aave fork has been updated, which means redeploying a new `AaveV3PreliquidationFallbackHandler` contract with new parameters.

Issue L-14: Typos and unused code in `MigrationHelper`

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/53

Vulnerability Detail

The following typos happen in `MigrationHelper`:

- `_perpareWithdrawCalls` should be `_prepareWithdrawCalls`.
- `_perpareDepositCalls` should be `_prepareDepositCalls`.

And the following code at the end of the loop in `_performMigration` is unused:

```
borrowBalance = borrowBalance >= borrowBalanceBatch ? borrowBalance -
↪  borrowBalanceBatch : 0;
lendBalance = lendBalance >= lendBalanceBatch ? lendBalance - lendBalanceBatch : 0;
```

Impact

Code is not clean.

Code Snippet

Tool Used

Manual Review

Recommendation

Minor code cleanup

Issue L-15: Migrations can be delayed by making `WithdrawManager` own shares.

Source: https://github.com/sherlock-audit/2025-10-superlend-hyperevm-expansion-oct-1st/issues/56

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

When a V1 vault is migrated, the `migrate` function ensures that there are no ongoing withdrawal requests by checking that the `WithdrawManager` contract does not hold any vault shares:

```
// Ensure withdraw manager has no balance to prevent conflicts
address blackListedUser = ISuperloopLegacy(oldVault).withdrawManagerModule();
uint256 blackListedUserBalance = ISuperloop(oldVault).balanceOf(blackListedUser);
if (blackListedUserBalance > 0) {
    revert("withdraw manager must be empty");
}
```

An attacker can exploit this check to delay migration by depositing into the vault on behalf of the `WithdrawManager` contract. This results in a non-zero share balance for the withdraw manager, blocking the migration until admins intervene.

Impact

Migration can be delayed by a griefing attack.

Code Snippet

Tool Used

Manual Review

Recommendation

No changes to the migration code are necessary.

If the attack is executed, administrators can call `setWithdrawManagerModule` to assign a new withdraw manager that does not hold any vault shares. Before doing so, it's critical to verify that the original withdraw manager has no pending legitimate withdrawals for users.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.